



**Instituto Tecnológico de Mazatlán**



## **Comunicación I2C entre 4 Arduinos en Arduino UNO R4 WiFi**

**Carrera:**

**Ingeniería en Sistemas Computacionales**

**Materia:**

**Sistemas Programables**

**Profesor:**

**Miguel Hernandez Barrón**

**Integrantes:**

**Daniel Bustamante Gamboa**

**Iván Luna Hernández**

## 1. Objetivo

Comprender el funcionamiento del bus I2C mediante la comunicación entre un Arduino maestro y tres Arduinos esclavos, que comparten las líneas SDA y SCL y se identifican con direcciones distintas (0x08, 0x09 y 0x0A). El alumno debe ser capaz de armar el bus, asignar direcciones y programar el envío y la recepción de datos entre los distintos nodos del bus.

## 2. Descripción de la práctica

Se arma en Tinkercad un bus I2C con cuatro Arduino Uno. El maestro se comunica con los esclavos a través de las líneas SDA (A4) y SCL (A5), con tierra (GND) común entre todos los dispositivos. Cada esclavo tiene un rol y una dirección distinta dentro del bus:

| Nodo      | Dirección | Función                                                                   |
|-----------|-----------|---------------------------------------------------------------------------|
| Esclavo 1 | 0x08      | Recibe una orden del maestro y enciende/apaga un LED.                     |
| Esclavo 2 | 0x09      | Recibe un ángulo del maestro y mueve un servomotor.                       |
| Esclavo 3 | 0x0A      | Lee un potenciómetro y envía su valor al maestro cuando este lo solicita. |

Cada 500 ms el maestro pide el valor del potenciómetro al esclavo 3, lo convierte a un ángulo de 0° a 180° y se lo envía al esclavo 2. Además, por el Monitor serie se puede escribir 1 o 0 para encender o apagar el LED del esclavo 1. El maestro muestra en el Monitor serie el valor leído y avisa si algún esclavo no responde.

## 3. Material utilizado

### Hardware (simulado en Tinkercad):

- 4 Arduino Uno R4 WiFi
- 1 protoboard (Breadboard Small)
- 1 LED
- 1 resistencia de 220  $\Omega$  (470  $\Omega$  en la versión adaptada al R4)
- 1 micro servo
- 1 potenciómetro
- Cables de conexión
- Opcional: 2 resistencias de 4.7 k $\Omega$  (pull-up en SDA y SCL)

### Software:

- Librerías Wire y Servo (incluidas en el entorno de Arduino)
- Monitor serie

## 4. Diagrama de conexión

Las cuatro placas comparten las líneas SDA (A4) y SCL (A5) del bus I2C, así como una tierra (GND) común. El LED con su resistencia y el potenciómetro se conectan a la protoboard, y el servo se alimenta y controla desde el esclavo correspondiente. Cada placa se alimenta de forma independiente por su propio puerto USB, sin unir las líneas de 5V entre ellas.

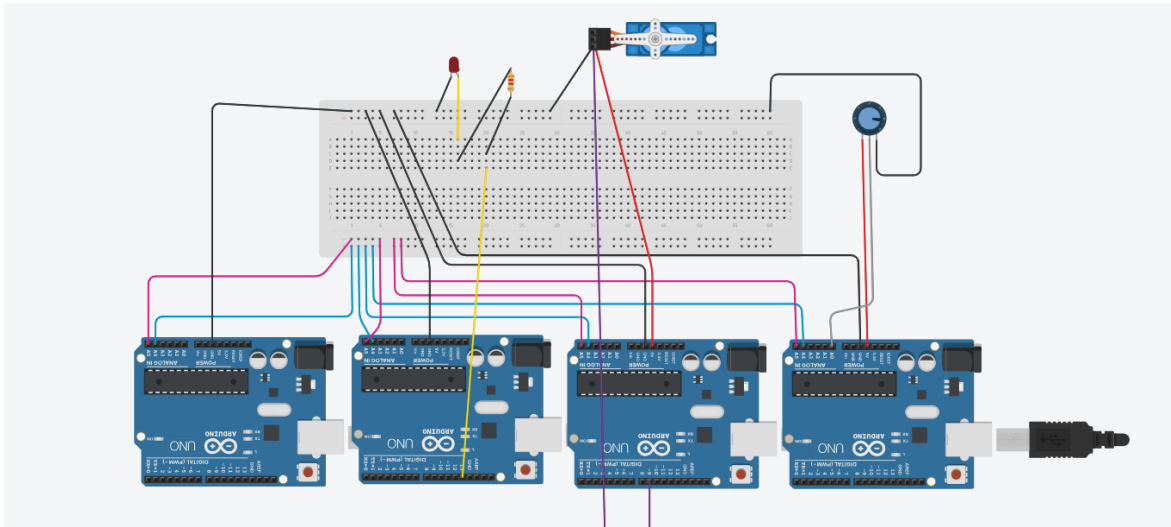


Figura 1. Diagrama esquemático del circuito (Tinkercad): 4 Arduino Uno, LED, servomotor y potenciómetro conectados al bus I2C.

## 5. Código fuente

A continuación se presenta el código correspondiente a cada nodo del bus, en su versión final adaptada para el Arduino Uno R4 WiFi.

### 5.1 Maestro

```
#include <Wire.h>

const uint8_t DIR_LED = 0x08; // Esclavo 1
const uint8_t DIR_SERVO = 0x09; // Esclavo 2
const uint8_t DIR_POT = 0x0A; // Esclavo 3

unsigned long ultimaLectura = 0;

void setup() {
  Serial.begin(9600);
  while (!Serial && millis() < 3000) { } // espera el USB del R4 (max. 3 s)
  Wire.begin(); // sin direccion = maestro
  Serial.println("Maestro listo. Escribe 1 (LED encendido) o 0 (LED apagado)");
}

void loop() {
  // --- Orden por el Monitor serie: enciende/apaga el LED del esclavo 1 ---
  if (Serial.available() > 0) {
    char c = Serial.read();
    if (c == '1' || c == '0') {
      byte orden = (c == '1') ? 1 : 0;
      Wire.beginTransmission(DIR_LED);
      Wire.write(orden);
      if (Wire.endTransmission() != 0) {
        Serial.println("Esclavo 1 (LED) no responde");
      }
    }
  }

  // --- Cada 500 ms: pide el potenciómetro al esclavo 3 y mueve el servo ---
  if (millis() - ultimaLectura >= 500) {
    ultimaLectura = millis();
    if (Wire.requestFrom(DIR_POT, 2) == 2) {
      byte alto = Wire.read();
      byte bajo = Wire.read();
      int valor = (alto << 8) | bajo; // 0 a 1023
      byte angulo = map(valor, 0, 1023, 0, 180);

      Wire.beginTransmission(DIR_SERVO);
      Wire.write(angulo);
      if (Wire.endTransmission() != 0) {
        Serial.println("Esclavo 2 (servo) no responde");
      }

      Serial.print("Potenciómetro: ");
      Serial.print(valor);
      Serial.print(" -> servo a ");
      Serial.print(angulo);
      Serial.println(" grados");
    } else {
      Serial.println("Esclavo 3 (potenciómetro) no responde");
    }
  }
}
```

### 5.2 Esclavo 1 — LED (0x08)

```
#include <Wire.h>

const uint8_t PIN_LED = 13;
```

```

void setup() {
  pinMode(PIN_LED, OUTPUT);
  Wire.begin(0x08); // direccion de este esclavo
  Wire.onReceive(recibir);
}

void loop() {
}

void recibir(int cuantos) {
  while (Wire.available()) {
    byte dato = Wire.read();
    digitalWrite(PIN_LED, dato ? HIGH : LOW);
  }
}

```

### 5.3 Esclavo 2 — Servo (0x09)

```

#include <Wire.h>
#include <Servo.h>

Servo miServo;
volatile byte angulo = 90;
volatile bool hayDato = false;

void setup() {
  miServo.attach(9);
  miServo.write(90);
  Wire.begin(0x09);
  Wire.onReceive(recibir);
}

void loop() {
  // El servo se mueve aqui, no dentro de la rutina de recepcion
  if (hayDato) {
    hayDato = false;
    miServo.write(angulo);
  }
}

void recibir(int cuantos) {
  while (Wire.available()) {
    int a = Wire.read(); // se lee UNA sola vez
    if (a > 180) a = 180;
    if (a < 0) a = 0;
    angulo = a;
    hayDato = true;
  }
}

```

### 5.4 Esclavo 3 — Potenciómetro (0x0A)

```

#include <Wire.h>

void setup() {
  Wire.begin(0x0A);
  Wire.onRequest(enviar);
}

void loop() {
}

void enviar() {
  int valor = analogRead(A0); // 0 a 1023
  Wire.write(highByte(valor)); // se manda en 2 bytes
  Wire.write(lowByte(valor));
}

```

## 6. Ajustes necesarios para el Arduino Uno R4 WiFi

El código original del maestro y de los esclavos fue escrito pensando en el Arduino Uno R3. Al migrar a la placa R4 WiFi fue necesario considerar los siguientes cambios:

- 1. Monitor serie:** en el R4, el puerto serie es USB nativo y los primeros mensajes se pierden si el Monitor abre tarde. Se agregó al maestro una línea que espera a que se abra el puerto (`while (!Serial && millis() < 3000)`).
- 2. Resistencia del LED:** en el R4 se usa 470  $\Omega$  en lugar de 220  $\Omega$ , ya que sus pines soportan menos corriente que los del R3 (del orden de 8 mA). Con 220  $\Omega$  el LED pediría cerca de 14 mA.
- 3. Pull-ups:** en montaje físico se agrega una resistencia de 4.7 k $\Omega$  de SDA a 5V y otra de SCL a 5V, ya que en el R4 estas resistencias ya no son opcionales, porque las placas no traen pull-ups internas fuertes.
- 4. Tierra y alimentación:** se unen los GND de las cuatro placas, pero no se conectan los 5V entre ellas. Cada placa se alimenta por su propio puerto USB.
- 5. Placas necesarias:** se requieren 4 placas por equipo. Con una placa por alumno, lo normal es formar equipos de 4 personas.

## 7. Errores frecuentes identificados

Durante la práctica se identificaron dos errores comunes al trabajar con este montaje:

- **Resistencia con ambas patas en la misma columna de la protoboard:** los huecos de una columna están unidos por dentro, por lo que la corriente pasa sin resistencia (53.5 mA) y el LED se quema. Las patas de la resistencia deben ir en columnas distintas.
- **Leer varias veces el mismo dato de I2C dentro de una misma expresión:** el dato se consume en la primera lectura y las siguientes devuelven un valor inválido. El dato debe leerse una sola vez y guardarse en una variable antes de usarlo.

## 8. Conclusiones

1. El bus I2C permite comunicar varios dispositivos usando solo dos líneas (SDA y SCL) más tierra común. Agregar un esclavo no requiere pines adicionales, solo una dirección diferente.
2. Cada esclavo debe tener una dirección única. Si dos comparten la misma dirección, ambos responden al mismo tiempo y los datos se corrompen.
3. El maestro controla la comunicación: decide con quién habla, cuándo y si pide o manda datos. El esclavo solo responde cuando se le llama.
4. La comprobación del resultado de `endTransmission()` permite detectar cuando un esclavo no contesta (NACK), en lugar de que el sistema se congele.
5. Se usó `millis()` en lugar de `delay()` en el maestro, de modo que atiende al Monitor serie y consulta al potenciómetro sin bloquearse.

6. Se identificaron dos errores frecuentes de montaje y de manejo del bus (ver sección 7), cuya corrección fue clave para el correcto funcionamiento del sistema.
7. Cada Arduino de la simulación necesita su propio programa, y los cuatro deben compartir la misma tierra (GND).